Docker Compose

This document describes the Docker-based prototype deployment for DxAssist.

The prototype uses a single Compose file from the repository root:

```
compose.yaml
```

Services

The prototype stack starts the following services:

- `db`: PostgreSQL 17 database used by the backend.
- `backend`: Django API from `./dxassist-backend`.
- `frontend`: Next.js application from `./dxassist-frontend`.
- `scheduler`: orchestration service from `./dxassist-scheduler`.
- `dxassist-angiography`: demonstration angiography module from `./models/dxassist-angiography`.
- `dxassist-screening`: demonstration blood-test module from `./models/dxassist-screening`.

All services run on the shared `dxassist-network` Docker network. The backend waits for a healthy database before starting. The scheduler depends on the two demonstration model services.

Exposed Local Addresses

After startup, the main local addresses are:

- Frontend: `http://localhost:3000`
- Backend API: `http://localhost:8000/api/`
- Django admin: `http://localhost:8000/admin/`
- PostgreSQL: `localhost:5432`
- Scheduler TCP port: `localhost:8001`
- Angiography module TCP port: `localhost:8002`
- Blood-test module TCP port: `localhost:8003`

The browser talks to the backend through `localhost:8000`. Inside Docker, services talk to each other by service name, for example `backend`, `scheduler`, `dxassist-angiography`, and `dxassist-screening`.

Required Environment

Create a root `.env` file from `.env.example` before starting the stack:

```
cp .env.example .env
```

For local prototype work, the defaults are usually enough. The most important values are:

```
SECRET_KEY=change-me
POSTGRES_DB=dxassist
POSTGRES_USER=dxassist
```

```
POSTGRES_PASSWORD=change-me
BACKEND_PORT=8000
FRONTEND_PORT=3000
POSTGRES_EXPOSED_PORT=5432
BACKEND_URL=http://backend:8000
```

The committed `.env.example` is safe to share. The local `.env` file is ignored by git and should contain machine-specific values and local secrets.

Start The Prototype

Run all commands from the repository root.

Build and start the full prototype stack:

```
docker compose up --build
```

To keep the stack running in the background:

```
docker compose up --build -d
```

The backend container runs database migrations automatically during startup.

First Local Login

There is no public registration flow. Create an administrator account from the running backend container:

```
docker compose exec backend python manage.py createadmin \
  --email admin@example.local \
  --name Admin \
  --password change-me
```

Then open the frontend at:

```
http://localhost:3000
```

Use the created email and password to log in.

Typical Prototype Flow

Once the stack is running:

1. Open `http://localhost:3000`.
2. Log in with an admin-created account.
3. Select one of the diagnostic modules.
4. Upload the required input file or files.
5. Start the analysis.
6. Review the generated demonstration report.

The current diagnostic modules are demonstration modules. They return fixed example values and do

not perform real medical analysis.

Useful Commands

Show running services:

```
docker compose ps
```

Follow logs for all services:

```
docker compose logs -f
```

Follow logs for one service:

```
docker compose logs -f backend
docker compose logs -f scheduler
docker compose logs -f frontend
```

Run backend checks manually:

```
docker compose exec backend python manage.py check
```

Open a backend shell:

```
docker compose exec backend python manage.py shell
```

Rebuild after dependency or Dockerfile changes:

```
docker compose build
docker compose up
```

Stop The Prototype

Stop containers while keeping volumes:

```
docker compose down
```

This preserves local PostgreSQL data and frontend cache volumes.

Reset Local Prototype Data

Use this when you want a clean local state or when debugging Compose, Dockerfile, dependency, or volume issues:

```
docker compose down --volumes --rmi local --remove-orphans
docker compose up --build
```

This removes named volumes, locally built images, and orphaned containers. It also wipes local

PostgreSQL data and frontend cache volumes.

Reset Only Frontend Volumes

If the Next.js cache or frontend dependencies get into a bad local state:

```
docker compose down
docker volume rm dxassist_frontend_node_modules dxassist_frontend_next
docker compose up --build
```

Common Troubleshooting

If the frontend cannot reach the backend, check that:

- the backend is available at `http://localhost:8000/api/`,
- `BACKEND_PORT` is not already used by another local process,
- `CORS_ALLOWED_ORIGINS` includes `http://localhost:3000`,
- the frontend container has `BACKEND_URL=http://backend:8000`.

If analysis fails because the scheduler is unavailable, check that:

- the `scheduler` container is running,
- the scheduler can reach `dxassist-angiography:8002`,
- the scheduler can reach `dxassist-screening:8003`,
- the backend is configured to reach the scheduler at `scheduler:8001`.

If database startup fails, check that:

- the `db` container is healthy,
- `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD` match between Compose and the backend,
- no other local PostgreSQL instance is already using the exposed port.

Machine-Wide Docker Cleanup

Rarely needed:

```
docker system prune -a --volumes
```

This removes stopped containers, unused images, unused volumes, and unused networks across the whole machine. Use it only when you are comfortable removing Docker resources outside this project too.